



BookIt

Appointment Booking System

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the folders fit together.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

A monorepo using npm workspaces: a Vite React SPA (`client/`) and an Express + `pg` API (`server/`), run together with `concurrently` . The API route handlers are the backend.

```
Appointment Booking System/ (npm workspaces)
├─ package.json           # root scripts: dev, db:setup...
├─ server/
│   ├─ src/index.ts       # Express entry
│   ├─ src/config.ts      # dotenv config
│   ├─ src/db/            # pool, schema.sql, migrate, seed
│   ├─ src/routes/        # public.ts, admin.ts
│   ├─ src/services/      # slots, booking, email
│   └─ src/middleware/    # auth, errors
└─ client/
    ├─ src/App.tsx api.ts types.ts format.ts
    ├─ src/components/Layout.tsx
    ├─ src/pages/         # customer pages
    └─ src/admin/         # admin pages
```

The one rule: the client never touches the database. It calls same-origin `/api/...` (Vite proxies to Express :4000); Express validates with Zod and talks to PostgreSQL through a shared `pg.Pool` . Admin requests carry a `Bearer` JWT from localStorage.

1 • Server (server/src/)

ENTRY

src/index.ts

Express app: CORS (origin = client), JSON parsing, `/api/health` DB check, mounts `publicRouter` at `/api` and `adminRouter` at `/api/admin` (behind `requireAdmin`), and a centralized error handler.

CONFIG

src/config.ts

dotenv-backed config (port, DATABASE_URL, JWT secret, client origin, SMTP settings).

DB POOL

src/db/pool.ts

The shared `pg.Pool`.

SCHEMA ★

src/db/schema.sql

The full PostgreSQL schema: users, providers, services, schedules, breaks, time_off, customers, bookings, booking_events — plus the `btree_gist` extension, the custom `timerange` type, and the **GiST exclusion constraint** that prevents overlapping bookings.

MIGRATE / SEED

src/db/migrate.ts • src/db/seed.ts

`migrate` creates the DB if missing and applies `schema.sql` idempotently. `seed` wipes and inserts the demo admin, 6 providers (2 doctors, 2 salons, 2 turfs) with services/schedules/breaks, and 3 sample bookings.

ROUTES

src/routes/public.ts • admin.ts

public: `/auth/login`, `/business-types`, `/providers`, `/providers/:id`, `/providers/:id/slots`, `POST /bookings`, `/bookings/lookup`, `/bookings/:code/cancel`. **admin** (JWT): `/stats`, `/bookings` (+filters, `/:id/events`, `/:id/status`), `/day`, provider CRUD, schedule, time-off, service CRUD.

AVAILABILITY ENGINE

src/services/slots.ts

`computeSlots()` generates bookable slots by walking working windows in `slot_step_min` steps and subtracting breaks/time-off/bookings/buffer, applying lead time + horizon.

TRANSACTIONAL BOOKING ★

src/services/booking.ts

`createBooking()` : advisory lock → re-validate slot with the same engine → upsert customer by email → insert booking → insert `created` audit event → commit; maps `23P01` / `23505` to a 409 `BookingConflictError`. Also the booking-code generator.

EMAIL · MIDDLEWARE

src/services/email.ts · middleware/auth.ts · errors.ts

nodemailer transport + HTML templates + dev outbox writer; `signAdminToken` / `requireAdmin` JWT middleware; `asyncHandler` + centralized error handler (Zod → 400, custom `status` → right code).

2 • Client (client/src/)

FILE	PURPOSE
<code>main.tsx</code> • <code>App.tsx</code>	React entry (BrowserRouter); route table — public routes under <code>Layout</code> , admin routes under <code>AdminLayout</code> .
<code>api.ts</code>	The fetch wrapper (<code>api.get/post/put/patch/del</code>), <code>ApiError</code> , and JWT token storage helpers (localStorage <code>bookit_admin_token</code>).
<code>types.ts</code> • <code>format.ts</code>	Shared TS types; formatting helpers (money ₹, time, dates, weekday labels).
<code>components/Layout.tsx</code>	Public site shell (header/nav/footer + <code><Outlet/></code>).
<code>pages/Home.tsx</code>	Landing: category cards + feature strip.
<code>pages/Providers.tsx</code>	Browse providers by category with search.
<code>pages/ProviderDetail.tsx</code>	Profile + the 3-step booking flow; handles 409 by refreshing slots.
<code>pages/Confirmation.tsx</code> • <code>Manage.tsx</code>	Post-booking confirmation (code); look-up-and-cancel by code+email.
<code>admin/AdminLogin.tsx</code> • <code>AdminLayout.tsx</code>	Login → store JWT; admin shell + auth guard.
<code>admin/Dashboard.tsx</code>	Metrics dashboard (today, 7-day, revenue, cancel rate, per-provider).
<code>admin/Bookings.tsx</code> • <code>DayView.tsx</code>	Bookings table + status actions + event history; day timeline across providers.
<code>admin/Providers.tsx</code> • <code>ProviderEdit.tsx</code>	Provider list; editor for details, policies, weekly schedule + breaks, time-off, and service CRUD.

3 • Commands

COMMAND	DOES
<code>npm install</code>	Install root + both workspaces.
<code>npm run db:setup</code>	Create DB + apply schema, then seed demo data.
<code>npm run dev</code>	Run API (:4000) + client (:5173) together.

Prerequisite & demo: PostgreSQL 13+ running (the migrate step creates the `appointments` DB and needs the `btree_gist` extension). Open `localhost:5173`; admin at `/admin` with `admin@bookit.local` / `admin123`.